

マルチエージェント・コード生成における セッション継続の費用

商用コーディングエージェント CLI の挙動測定と事前指定比較

The Economics of Session Affinity in Multi-Agent Code Generation:
A Measurement Study of a Commercial Coding-Agent CLI

2026 年 9 月 4 日 統合改訂稿 v0.2

要旨

関連するコーディング作業を同じ会話で続ければ、背景情報の再取得を減らせる可能性がある。一方、継承する履歴を処理する費用や、作業を同じ会話にまとめるための実行順制約も生じる。本研究では商用 CLI のキャッシュ挙動を測定し、利用者側の記録に基づく費用モデルを構成した。短いプロンプトでは履歴のキャッシュ読みを確認したが、初回の書き直しを利用者側から確実に制御する方法は確立できなかった。次に、1 つの Python リポジトリ内の 7 仕様を対象に、新規セッション方式 B と、編集範囲の重なりで作業をまとめて会話を継続する方式 C を、固定モデル・固定 CLI で 98 対、196 ラン比較した。事前指定した 20% 削減の達成条件は満たされず、対ごとの相対費用差の中央値は C で 5.51% 高かった。宣言済みの相対尺度へ実装を訂正した BCa 95% 区間は $[-3.18, +10.12]\%$ であり、今回の集約中央値に対する 20% 以上の削減とは整合しなかった。時間比は 1.0425、片側 90% 上限は 1.1335 で、許容上限 1.15 を下回った。受入全通過率は B が 98/98、C が 97/98 で、通過率差の計算を訂正した片側 90% 下限は -5.10 ポイントとなり、許容下限 -10 ポイントを上回った。モデルは 7 仕様中 6 仕様で費用比を相対誤差 25% 以内に予測したが、方向一致の条件は満たさなかった。1 つの競合仕様では C が全 14 対で高く、その仕様を除く参考集計の中央値は -1.4% だった。本結果は試した継続方式と課題構成に限定され、会話継続一般の優劣や、費用増加の単一の原因を確定するものではない。

キーワード：コーディングエージェント、セッション継続、プロンプトキャッシュ、費用計測、事前指定分析

Abstract

Continuing related coding tasks in one session may avoid repeated repository exploration, but also carries conversation history and imposes additional scheduling constraints. We characterize client-observable cache behavior in a commercial coding-agent CLI and construct and evaluate a cost model. Short probes confirm cached history reads; they do not establish a reliable client-side control for initial history rewriting. We compare fresh sessions (B) with write-scope-connected lane continuation (C) on seven task specifications within one Python repository, using a fixed model and CLI version (196 runs; 98 pairs). The pre-specified criterion of at least 20% cost reduction is not met. The median paired relative cost difference is +5.51%; after correcting the implementation to the declared relative scale, its BCa 95% interval is [-3.18, +10.12]%, inconsistent with a reduction of at least 20% in this aggregate estimand under the analysis assumptions. The runtime ratio is 1.0425 (one-sided 90% upper bound 1.1335; margin 1.15). Acceptance rates are 98/98 and 97/98; a corrected pass-rate-difference calculation gives a one-sided 90% lower bound of -5.10 percentage points, above the -10-point margin. Six of seven model predictions are within 25% relative error, but directional agreement fails. One contended specification is more expensive under C in all 14 pairs; excluding it gives a descriptive median difference of -1.4%. We preserve the original results and disclose the corrections. The findings concern the evaluated policy and workload mix, not session reuse in general or an isolated causal effect of serialization.

版と証拠の位置づけ。本稿は章別原稿を統合し、2026 年 9 月 4 日の照合で判明した分析実装・記述の不一致を訂正した版である。元のプロトコル、生データ、固定予測、旧分析結果は保持する。プロトコルは実験前の Git 履歴に保存されているが、独立した公開登録時刻は確認できていない。したがって本稿では「事前指定」と表記し、第三者登録機関による事前登録を主張しない。訂正箇所と再現方法を付録A、Dに示す。

目次

1 導入	5
2 背景と関連研究	5
2.1 商用 API のキャッシュと文脈再利用	5
2.2 複数エージェントの作業分割	6
2.3 推論基盤側の最適化との違い	6
2.4 事前指定と本稿の位置	6
3 キャッシュ挙動の測定と費用モデル	6
3.1 測定単位と用語	6
3.2 観測したことと、その限界	6
3.3 説明用の分解と損益分岐	7
3.4 固定した予測の実際の計算法	8

4 比較する実行方式	8
4.1 共通の制約とレーンの定義	8
4.2 直接の競合と、推移的な直列化	8
4.3 失敗・切替・中断の扱い	9
5 実装・実験方法	9
5.1 対象・仕様・規模	9
5.2 順序・時間間隔・キャッシュ共有の抑制	10
5.3 記録と費用・品質の定義	10
5.4 中断と停止条件	10
6 評価と結果	11
6.1 評価対象と事前指定の条件	11
6.2 改稿時に判明した実装不一致	11
6.3 費用：20% 削減の達成条件は満たされなかった	12
6.4 仕様別と CT 除外の感度分析	12
6.5 時間と受入テスト	13
6.6 モデル予測の一致と不一致	13
6.7 中心主張と追加分析	14
7 支持されなかった見立て	15
7.1 キャッシュ共有を単純な二択で捉えない	15
7.2 履歴の増大と書き直しの関係	15
7.3 予算付きレーンを成果としない	15
8 妥当性と適用範囲の限界	15
8.1 対象の狭さと仕様への依存	15
8.2 複合介入と観測できない状態	15
8.3 順序・運用変更と統計上の仮定	16
8.4 モデル近似と較正の量	16
8.5 品質の範囲と訂正の位置づけ	16
8.6 装置を再配布しないことによる再現の非対称	16
8.7 非盲検と公開前の証拠	16
9 結論	17
A 分析訂正の履歴	17
B 機構測定の記録と未確定事項	18
C 追加分析の定義	18

D 再現方法と成果物

18

1 導入

複数の AI にコード作成を分担させると、個々の AI が同じリポジトリのファイル構成、関数、実装上の約束を調べ直すことがある。人間の仕事にたとえば、新しい担当者にも何度も背景を説明する状況に近い。同じ担当者に続けて仕事を頼めば説明を省けそうだが、言語モデルでは過去の会話やツール出力も次の入力に含まれ、その処理に費用がかかる。キャッシュはこの入力の単価を下げて、処理対象の文脈量をゼロにはしない。

本研究の問いは「会話が継続できるか」ではなく、「継続による再取得の節約が、履歴を引き継ぐ費用を上回るか」である。対象は推論サーバや KV キャッシュを変更できない商用コーディングエージェント CLI であり、利用者が選べる実行順とセッション継続を調べる。KV キャッシュとは、モデルが入力を処理した中間状態を再利用するための仕組みである。本稿が直接取得するのはその内部状態ではなく、CLI が報告する読み取り・作成トークン数と費用である。

既存研究は、キャッシュ方式の費用評価 [4]、文脈の保存と縮約 [6]、タスク分割による時間・品質・費用の改善 [8] を扱っている。本稿はこれらの先行成果を前提とし、特定 CLI での継続境界の測定、実測に基づくモデルの検証、編集対象の重なりを利用する 1 つの継続方式の比較を組み合わせる。キャッシュ計測そのものや、関連タスクをまとめる発想が新しいとは主張しない。

貢献は次の 3 点である。

1. 継続時の履歴読み、新規セッション間の共有の条件、固定文脈、初回書き直しについて、クライアント記録から分かる範囲と分からない範囲を整理する。
2. 再取得の節約と履歴の運搬費用を分解し、独立した B 側の較正データで固定した予測を本実験に照合する。
3. 新規方式 B とレーン継続方式 C を事前指定の基準で比較し、費用削減基準の未達、仕様別の違い、分析実装の訂正とその影響を報告する。

適応スケジューラや会話の最適な切り替え規則は開発・評価していない。観測結果を根拠に改善候補を挙げることに、その改善を実証することを区別する。

2 背景と関連研究

2.1 商用 API のキャッシュと文脈再利用

Lumer ら [4] は、商用 3 社の API について、全履歴、システムプロンプトのみ、動的なツール結果を除く方式を比較し、DeepResearch Bench 上で費用と最初のトークンまでの時間を評価している。本稿との差は、キャッシュの有無の一般的評価ではなく、コーディング CLI の新規／継続境界と、複数タスクをまとめる実行方式を対象とする点にある。

Santoni の Contextual Memory Virtualisation (CMV) [6] は、会話状態を有向非巡回グラフで管理し、保存、分岐、縮約を扱う。76 のコーディングセッションによる事例評価を含み、再構築する文脈の費用も論じる。その観察は本稿の動機に関連するが、CMV の縮約・分岐と本稿のレーン継続は異なる介入である。本稿の B/C 比較を、CMV への直接の反証として解釈しない。

2.2 複数エージェントの作業分割

Co-Coder[8] は、依存グラフの分割とスケジューリングにより、エージェント間の文脈転送と並列性の関係を扱う。28 課題で通過率、時間、API 費用を評価し、費用削減も報告している。したがって「既存の協調研究は費用を測っていない」という差別化は成り立たない。本稿ではタスク分割方式を最適化せず、既存タスクの宣言した編集範囲からレーンを構成し、その継続方式と CLI の挙動を測る。

2.3 推論基盤側の最適化との違い

SGLang[9] は RadixAttention による KV キャッシュ再利用などを実行基盤に組み込む。Parrot[3] は要求間の関係をサービス側へ伝え、アプリケーション全体の実行を最適化する。本稿の利用者はそれらの内部キャッシュ配置や要求処理機構を操作できない。同じ「親和性」という語を使っても、操作対象と利用できる観測量が異なる。本結果をサービング層における親和性と負荷分散の一般的な優劣へ拡張しない。

2.4 事前指定と本稿の位置

エージェント実験ではモデル、プロンプト、評価指標、結果を見た後の変更などの選択肢が多く、事前登録の必要性が論じられている [7]。本研究は判定規則と変更履歴を実験前から保存した。ただし公開された独立時刻証明は確認できず、実験中の運用変更と、分析後の実装訂正もある。そのため、事前に指定した部分と後から追加・訂正した部分を明示する。本稿の貢献は測定と比較の記録にあり、新しい最適化問題の計算量や最適解法を提示するものではない。

3 キャッシュ挙動の測定と費用モデル

3.1 測定単位と用語

セッションは、履歴を引き継いで再開できる会話の単位である。タスクは 1 件の開発作業、ランは 1 仕様を 1 方式で実行して受入判定する単位とする。1 タスクの CLI 呼び出しの内部で、モデルとツールの複数回のやり取り（ターン）が発生しうる。短い挙動確認をプローブと呼び、本実験の 196 ランとは区別する。

キャッシュに以前の入力を利用できる状態を **warm**、その読み取りが失われる状態を **cold** と呼ぶ。ただし、固定文脈だけが残る中間状態もある。読み取り比・作成比で機械判定した **ambiguous** を、都合よく **warm** や **cold** へ読み替えない。プローブは主に CLI 2.1.220、本実験は 2.1.247 であり、プローブの挙動が本実験の全呼び出しで同一だったとは仮定しない。

3.2 観測したことと、その限界

M1 の 3 反復では、継承履歴 45,124、45,123、45,122 トークンが、それぞれ同数の **cache read** として報告された。再開呼び出しの費用は 0.01438 USD で、直前の初期呼び出しとの費用比は中央値で約 1/11.1 だった。これは短いプローブ内の呼び出し比較であり、同じコーディング課題を解く B/C の比較ではない。

M7 では 500 文字の追加で書き直しが起き、16,000 文字では起きない例を得た。初期 **cache read** が少ない観測と書き直しが対応したが、提供側のキャッシュ状態を独立に操作し

測定	観測	解釈の範囲
M1：warm 再開	一意な入力接頭辞で 3 反復。履歴読み比は全て 1.0000	小さな追加要求で履歴のキャッシュ読みを確認。仕事全体の費用削減を意味しない。
M4：新規から新規	3 反復で入力 cache write/read 内訳が一致	この条件では共通ユーザー入力による追加割引を確認しなかった。全条件での非共有は主張しない。
固定文脈・交差共有	新規要求でも約 21.6k の cache read。継続後の別の新規要求で共有を観測	固定文脈の共有と、ユーザー履歴の条件付き共有を分ける。
M6：履歴増加	2 系列。後続の cache write は新規追加分でほぼ一定	継承履歴が増えると毎回の書き込みも必ず増える、という見立ては支持されない。
M7：追加量	500-16,000 文字の掃引と 2,000 文字の 6 反復	書き直しは追加量の単調な閾値では説明できない。初期の cache read との関連は観測的所見。
M2・M3：間隔／切替	長間隔 2 件、モデル切替 2 件の分類は ambiguous	正確な寿命や「モデル切替で全損」を確立していない。

表 1: 機構測定の要約。詳細ログと判定単位は付録Bに示す。k は千トークン。

ていない。したがって「状態が原因だと特定した」ではなく、「追加量だけでは説明できず、初期状態の指標と対応した」と述べる。事前の確実な制御は未確立でも、利用者側に全く情報がないわけではない。事後の観測に基づく適応は候補として残る。

3.3 説明用の分解と損益分岐

タスク j を新しく始めるときの再取得費用を e_j 、前タスク i から引き継ぐ履歴を H_i 、継続後のターン数を K'_j とする。基準入力単価を p_{in} 、キャッシュ読み・書きの単価係数を α_r, α_w とし、初回に履歴を書き直す指示変数を $\rho \in \{0, 1\}$ と置く。履歴の運搬費用は、単純化したモデルでは

$$c_{\text{carry}}(i, j) \simeq p_{\text{in}} \{ \rho \alpha_w + (K'_j - \rho) \alpha_r \} H_i \quad (1)$$

となる。本来の作業と固定文脈の費用が方式間で等しいと近似できるなら、継続が得になる条件は

$$c_{\text{carry}}(i, j) < e_j \iff H_i < H_j^* = \frac{e_j}{p_{\text{in}} \{ \rho \alpha_w + (K'_j - \rho) \alpha_r \}}. \quad (2)$$

e_j は USD、 H_i はトークン、 p_{in} は USD/トークンである。キャッシュが使えても、長い履歴を多数のターンで処理すれば費用が増えることを表す。

この式は意思決定の完成形ではない。継続はターン数、出力、再探索の順序も変えうるため、共通費用の相殺は近似にすぎない。また書き直しの指示変数を、追加プロンプトの大きさや公

称保持時間だけで確実に決めることはできない。再開間隔は利用者が動かせても、キャッシュ全体の状態は直接制御できない。

3.4 固定した予測の実際の計算法

モデル検証には、アーム B だけを実行した 7 仕様各 1 反復の較正データを用いた。編集ツール (Write、Edit、NotebookEdit、MultiEdit) が最初に現れる前を探索区間、以後を作業区間と定義する。各タスクの最小文脈量を $\hat{S}_j$ とし、探索ターン t でそれを超える文脈量の和を

$$\hat{E}_j = \sum_{t \in \mathcal{E}_j} \max(0, \text{context}_{jt} - \hat{S}_j) \quad (3)$$

とする。これは探索の意味を直接測った量ではなく、ツール名とトークン数による代理量である。

固定した実装 `analysis/calibrate.py` は、B 側の実測費用を土台とし、レーンの 2 件目以降を

$$\hat{K}'_j = \max(1, K_j - K_{E,j}), \quad (4)$$

$$\hat{C}_{C,j} = C_{B,j} - p_{\text{in}} \alpha_r \hat{E}_j + p_{\text{in}} \{ \rho \alpha_w + (\hat{K}'_j - \rho) \alpha_r \} \hat{H}_i \quad (5)$$

で予測する。ここで $\rho = 1$ に固定し、 $\hat{H}_i$ は直前タスクの B 側較正での最大文脈量を使う。レーン先頭は $\hat{C}_{C,j} = C_{B,j}$ で、仕様内を合算して $\hat{r}_s = \sum_j \hat{C}_{C,j} / \sum_j C_{B,j}$ を得る。説明用の式と実装の近似を区別し、実装が継続による累積履歴を逐次シミュレーションしているとは書かない。

較正実装の単価は、記録された `claude-sonnet-5` に対し 100 万トークン当たり入力 3.00、出力 15.00、cache read 0.30、cache write 3.75 USD である。これは当該予測コードの固定定数で、現在のサービス価格の案内ではない。出力費用を含む B 側実測費用を基礎にしているが、継続による出力の変化や別のキャッシュ作成料金は明示的に予測しない。予測ファイルは本実験前のコミット `4fbb713` に保存されており、本稿の訂正で再較正していない。

4 比較する実行方式

4.1 共通の制約とレーンの定義

タスク間の先行関係を守り、宣言された編集対象が直接重なるタスクを同時に実行しない。方式 B はこの実行中タスクとの直接の重なりを確認し、各タスクを新規セッションで呼ぶ。方式 C は同じ共通制約に加え、編集範囲が重なるタスクを辺で結んだ無向グラフの連結成分をレーンとする。連結成分とは、直接または他のタスクを経由してつながったまとまりである。同一レーンは同時に 1 タスクだけとし、2 件目以降を前のセッション ID で再開する。別レーン間の並列性は残す。

4.2 直接の競合と、推移的な直列化

依存関係がない 3 タスク A, B, C について、編集範囲 W_A, W_B, W_C が

$$W_A \cap W_B \neq \emptyset, \quad W_B \cap W_C \neq \emptyset, \quad W_A \cap W_C = \emptyset \quad (6)$$

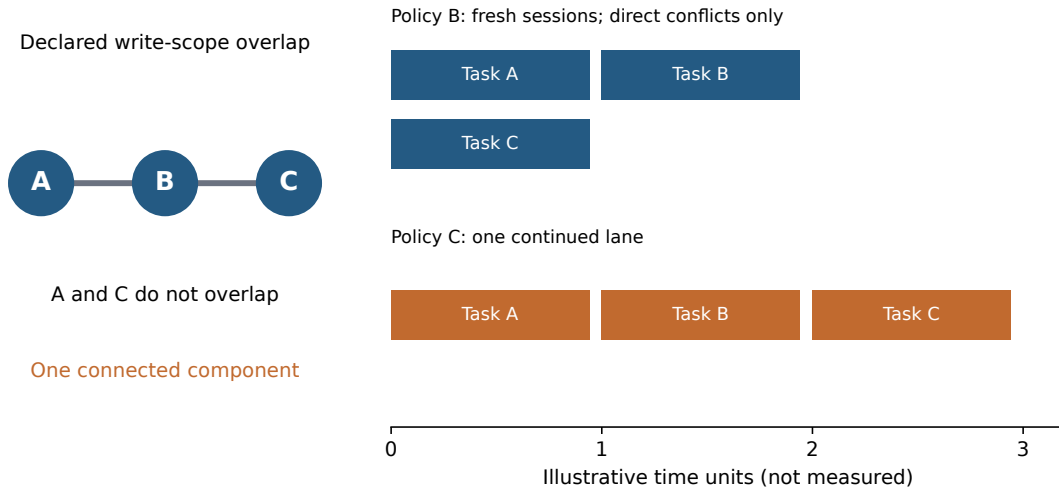


図 1: CT の構造を示す模式図。タスク A-B、B-C に編集範囲の重なりがあるが、A-C にはない。棒の長さは模式的で、実測の所要時間ではない。タスク名 A/B/C と方式名 B/C を区別する。

なら、B 方式は A と C を同時に実行できる。一方、C 方式では 3 タスクが同じ連結成分なので全て順番に実行する（図1）。「直接重なるタスクは同時に走れない」ことから「連結成分内は全部順番にしても追加の代償がない」とは導けない。

この比較は会話継続とレーン内直列化を同時に変える複合的な介入である。したがって方式 C の費用差を、直列化だけ、あるいはキャッシュだけの因果効果として分離できない。同じスケジュールで新規／継続だけを変える追加アームは、本研究では評価していない。

4.3 失敗・切替・中断の扱い

モデル不一致時にはセッションの継承をやめる処理がある。モデルを混在させる比較は本実験の対象外である。一方、実装を照合すると、タスク失敗を判定する前にも取得済みセッション ID が記録される。そのため「失敗時には必ずセッションを破棄する」という旧稿の説明は採用しない。先行タスク失敗時の依存先ブロックは行うが、失敗文脈からの継続を全経路で防ぐ保証は評価していない。ラン全体の受入不合格と、途中のタスク例外も区別する。レーン長の上限、適応的な分割、長い中断後のキャッシュ保持保証は実装・評価の成果として主張しない。

5 実装・実験方法

5.1 対象・仕様・規模

実行基盤は既存のマルチエージェント・フレームワーク、実験対象は Python の square-media リポジトリである。7 つの独立したリポジトリを比較したわけではない。対象を、受入テストがあり実装が未完成的な 3 つのコミット（d4449a0、3be3676、6f5b695）に固定した。各ランは別の Git worktree で実行し、他ランの生成物を引き継がない。

各仕様を 2 方式で 14 反復し、98 対・196 ランを取得した。反復数はパイロットと事前指定の手順に基づく。本実験の期間は 2026 年 8 月 28 日から 9 月 2 日で、記録されたモデル ID は claude-sonnet-5、CLI は 2.1.247 である。モデル名は当該ログの識別子として記載する。最終データセットに欠測対・事後の外れ値除外はないが、その完成までに中断・再実行は

仕様	形状	レーン内のタスク数	役割
w_two_files	W	1 + 1	継続対象がない対照
n_db_chain	N	2	依存関係ですでに直列
m_mixed	M	2 + 1 + 1	依存鎖と独立作業の混合
s17_w_two_files	W	1 + 1	別系列の対照
s17_n_db_chain	N	2	別系列の依存鎖
s17_m_mixed	M	2 + 1	別系列の混合
ct_library	CT	3	依存なし・編集範囲の推移的な重なり

表 2: 7 仕様の構成。W の差はセッション再利用の効果ではなく、独立に実行したランの変動も含む。

発生した。

5.2 順序・時間間隔・キャッシュ共有の抑制

build_schedule は乱数種 20260801 で仕様順を並べ替え、奇数反復を B 先行、偶数反復を C 先行にする。アーム順そのものを各対で無作為抽出する設計ではない。保存された終了時刻から wall_s を引いて開始時刻を復元すると、B 先行 49 対、C 先行 49 対で、全 98 対が計画の先行方式と一致した。

実験途中に、全仕様の片方式を先に実行する順序から、仕様ごとに両方式を隣接して実行する順序へ変更した (A-5)。利用上限で比較の片側だけが残る状況への対応であり、費用判定の閾値を変更したものではない。実際の先行ラン終了から後続ラン開始までの間隔は中央値 0.27 秒、最大 38.24 分で、10 分以上離れた対は 11 対だった。終了時刻と丸めた所要時間からの推定なので、サブ秒値は精密な待機時間の測定ではない。

各ランのプロンプトに固有文字列 (nonce) を置き、別ランのユーザー入力接頭辞が一致しにくいようにした。同一ラン内ではその文字列を維持する。この措置は固定システム文脈の共有やサービス側の時間変動を除去しない。順序の均衡だけでキャッシュ状態の交絡がなくなったとはせず、離隔対を除く感度分析も報告する。

5.3 記録と費用・品質の定義

CLI 呼び出しごとの calls.jsonl に、タスク ID、セッション ID、継続の有無、モデル、CLI 版、入出力とキャッシュのトークン数、費用、ターン数を保存した。--output-format stream-json --verbose から取得したターン情報も残す。総費用は提供側が報告した USD 費用をラン内で合計した値であり、請求書との独立照合はしていない。訂正時にはデータセットと run.json の費用が一致することを確認した。

時間はランの投入から受入判定までの経過時間であり、モデルの推論時間だけではない。品質は受入テストが全て通ったランを 1、それ以外を 0 とする二値である。個々のテストの平均通過率、コードの保守性、人間による品質採点はこの指標に含まれない。宣言した編集範囲と実際の書き込みの一致率、範囲逸脱、LLM 審判による採点は測定していない。

5.4 中断と停止条件

旧集計はインフラ中断イベント 121 件を 196 ランで割り、61.7% として 20% の停止条件を検出した。A-6 は、繰り返し中断された同一ランをまとめると 27/196 (13.8%) であると記

録し、比較値の判定前に再開を決めた。本稿は A-6 の根拠と、旧集計が停止を検出した事実の両方を残す。イベント比率とラン比率は異なる指標であり、どちらかを無説明で消さない。利用上限、ローカル遮断の誤検知、worktree 不整合、実行順変更などの運用上の介入があり、その個別の影響を分離評価していない。

6 評価と結果

6.1 評価対象と事前指定の条件

対 i の相対費用差を $d_i = (C_i - B_i)/B_i$ とする。負なら C が安い。主要な点推定は全 98 対の $\text{median}(d_i)$ であり、仕様別中央値の中央値ではない。同数反復の 7 仕様を含む固定構成の集約値として解釈する。

仮説	問い・推定対象	事前指定の達成条件
H1 費用	対ごとの相対差、中央値	両側符号反転検定 $p < 0.05$ 、BCa 95% 区間が 0 を跨がない、かつ中央値 $\leq -20\%$
H2 時間	$\text{median}(T_C)/\text{median}(T_B)$	片側 BCa 上限 < 1.15 。実装は 90% を採用
H3 受入	$p_C - p_B$	片側 90% 下限 > -10 ポイント
H4b 予測	仕様別の総費用比 r_s	予測との相対誤差 25% 以内が過半数、かつ全仕様で優劣の方向一致

表 3: 判定の要約。H1 の相対尺度は分析前文書の決定 A に基づく。H2/H3 の許容差は運用上の判断値である。

符号反転検定の統計量は平均、費用の実質性と区間の対象は中央値である。再標本数 10,000、乱数種 20260802 とし、検定の p 値は $(b+1)/(10,000+1)$ で計算する [5]。BCa はバイアス補正と加速度を用いるブートストラップ区間である [2]。検定と区間が異なる統計量を扱うため、 $p < 0.05$ でも中央値の区間が 0 を跨ぐことは矛盾ではない。独立性と符号交換可能性を仮定した分析であり、決定的な交互順序から無作為化推論が自動的に成立するとはしない。

6.2 改稿時に判明した実装不一致

旧 E2 の H1 は、相対差を主とする分析前文書に反してドル差を検定・区間へ渡していた。H3 は通過率差の区間を求める際、関数の既定の中央値を使っていた。後者では 98 対中 97 対が差 0 のため、中央値の分布が 0 に集中し、下限 0.00 ポイントが出力された。これは通過率差の区間ではない。

旧コードと旧 JSON は変更せず、訂正コード analysis/revision_audit.py で H1 を相対差、H3 を平均差へ明示的に修正した。元の E1/E2 は一時ディレクトリへの再実行で JSON 全体の一致を確認し、凍結対象 398 ファイルのハッシュ一致も確認した。訂正は結果を知った後の作業であり、その日時と影響を隠さない。

指標	結果
相対費用差の中央値	+5.51%
訂正後の相対差平均による検定	$p = 0.0013$
訂正後の相対差中央値の BCa 95% 区間	$[-3.18, +10.12]\%$
C が安い対	44/98
総費用 B / C	58.35 / 62.91 USD
総費用の増加率	+7.82%
旧ドル差の検定	$p = 0.0047$
旧ドル差中央値の BCa 95% 区間	$[-0.0173, +0.0581]$ USD

表 4: 全体の費用結果。総費用の比と対ごとの比の中央値は別の統計量である。

6.3 費用：20% 削減の達成条件は満たされなかった

訂正後も中央値は削減方向ではなく、95% 区間は 0 を跨ぐため、H1 の複合的な達成条件は満たされない。一方、区間の下端は -20% より上にあるため、分析上の仮定の下では、今回の集約中央値について 20% 以上の削減とは整合しない。これは「効果が全くない」「全ての仕様で削減できない」という結論ではない。図2に、値が広く分散し、C が安い比較も多数あることを示す。

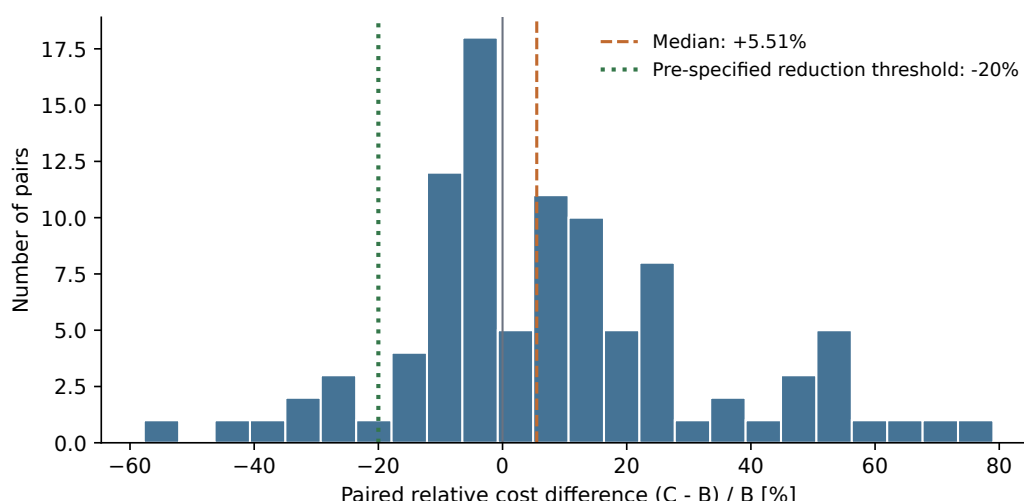


図 2: 全 98 対の相対費用差。破線は中央値、点線は 20% 削減の達成基準。各比較の変動と集約結果を区別する。

6.4 仕様別と CT 除外の感度分析

ct_library では相対費用差中央値が +50.6% で、C が全 14 対で高かった。他の 6 仕様では相対差中央値が -7.9% から +12.1% に分布した。CT を除く副次集計 84 対では中央値 -1.4%、訂正後の相対尺度の $p = 0.4950$ 、BCa 95% 区間 $[-5.10, +6.44]\%$ だった。旧稿の $p = 0.90$ はドル差による値であり、相対尺度の結果と混在させない。

全体の中央値の符号が仕様構成に敏感であることは言えるが、CT を除いた値を主要な結果へ差し替えない。CT は 1 仕様なので、形状一般の効果とその具体的な課題内容を分離できない。図3では全観測を仕様別に示す。

仕様	B 中央値	C 中央値	相対差中央値	C が安い対
s17_w_two_files	0.5547	0.5293	-7.9%	10/14
n_db_chain	0.4323	0.4467	-5.6%	10/14
s17_n_db_chain	0.4501	0.4790	-1.4%	8/14
s17_m_mixed	0.7451	0.7907	+5.0%	5/14
m_mixed	0.8054	0.8501	+5.2%	6/14
w_two_files	0.3609	0.3722	+12.1%	5/14
ct_library	0.6617	1.0255	+50.6%	0/14

表 5: 仕様別の費用（USD）と対ごとの相対差。B/C の中央値同士から相対差中央値を再計算することはできない。各仕様 14 対。

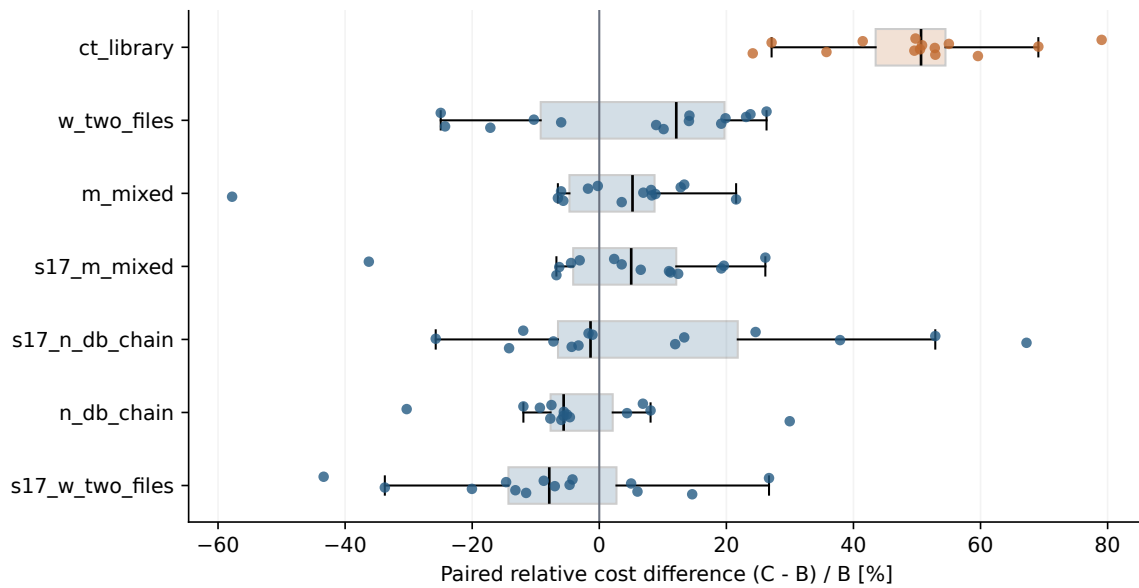


図 3: 仕様別の全観測と箱ひげ図。CT の 14 対は全て 0 より大きい。他仕様では正負の値が混在する。

6.5 時間と受入テスト

時間の中央値は B が 280.85 秒、C が 292.80 秒で、比は 1.0425 だった。対応を保ったブートストラップによる片側 90% 上限 1.1335 は 1.15 を下回り、指定した時間の非劣性基準を満たした。これは時間が同じという証明ではなく、採用した 15% の許容差についての判断である。

受入全通過は B が 98/98、C が 97/98 で、差は-1.02 ポイントだった。通過率差として平均を用いた訂正後の片側 90%BCa 下限は-5.10 ポイントで、-10 ポイントの基準を満たす。この許容差は比較的広く、1 件しか不一致がないデータから品質全般の同等性を保証しない。補助確認として、B 成功・C 失敗の確率の片側 90%Clopper-Pearson 上限を反転した保守的な通過率差下限は-3.91 ポイントだった（付録C）。

6.6 モデル予測の一致と不一致

仕様 s の実測比は $r_s = \sum_i C_{si} / \sum_i B_{si}$ であり、表5の相対差中央値とは異なる。大きさの条件は 6/7 で満たすが、方向一致は成立しない。旧コードは「 $r > 1$ か」という二値比較で方向を判定し、等値を減少側と同じ分類にしていた。その規則での不一致は n_db_chain と

仕様	実測 r_s	予測 $\hat{r}_s$	相対誤差	25% 以内
ct_library	1.4909	1.5575	-4.3%	はい
m_mixed	1.0060	1.1806	-14.8%	はい
n_db_chain	0.9571	1.5251	-37.2%	いいえ
s17_m_mixed	1.0205	1.2987	-21.4%	はい
s17_n_db_chain	1.0736	1.3730	-21.8%	はい
s17_w_two_files	0.9021	1.0000	-9.8%	はい
w_two_files	1.0334	1.0000	+3.3%	はい

表 6: 固定した予測との比較。相対誤差は $(r_s - \hat{r}_s)/\hat{r}_s$ 。等値の予測 1.0000 は増加の予測ではない。

w_two_files である。増加・等値・減少を分ける 3 値の記述では、それらに s17_w_two_files も加わる。旧稿の「2 仕様とも値下がりを実測できなかった」という説明は、旧コードの不一致集合と一致していなかった。どちらの扱いでも全仕様の一致を満たさず、H4b 不成立の結論は変わらない。

予測が実測を上回るのは 7 仕様中 6 仕様であり、全 7 仕様ではない (図4)。この方向の偏りは、較正が各仕様 1 反復、 $\rho = 1$ 固定、探索区間の代理量、累積履歴の近似などを点検する理由になる。ただし、偏りの原因や補正可能性を確立したわけではない。比が 1 に近ければ大きさが近くても方向を外すため、予測誤差 25% 以内を意思決定の正しさと同一視しない。

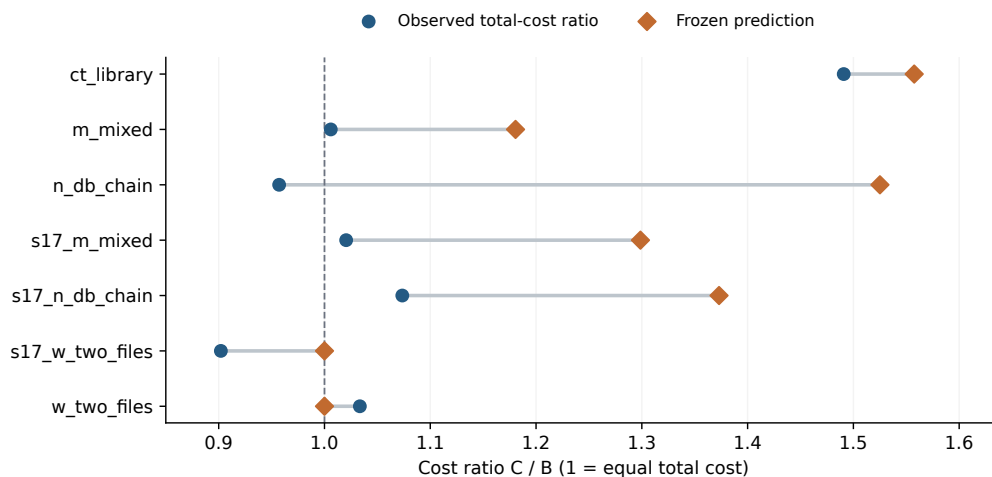


図 4: 予測と実測。橙の予測点が青の実測点より右にある過大予測は 6/7 仕様。破線の 1.0 は両方式の総費用が等しい境界。

6.7 中心主張と追加分析

「時間と品質を許容範囲に保ち、費用を 20% 以上削減する」という中心主張は、全ゲートを満たした場合だけ成立する。訂正前後とも費用の条件を満たさず、中心主張は支持されない。時間と受入の非劣性のみを、マージン付きで報告する。

改稿時の追加分析として、7 仕様それぞれの 14 対を仕様内で復元抽出するブートストラップを行った。プールした中央値の percentile 95% 区間は $[-2.17, +8.84]\%$ だった。仕様ごとの反復構成を保っても、この分析の範囲では 20% 削減と整合しない。これは固定 7 仕様条件づけた追加分析であり、リポジトリ母集団の区間ではない。10 分以上離れた 11 対を除く 87 対でも中央値は $+6.0\%$ 、訂正した BCa 区間は $[-3.27, +8.95]\%$ で、費用削減の達成条

件を満たさなかった。

7 支持されなかった見立て

7.1 キャッシュ共有を単純な二択で捉えない

当初は、別セッションで同じ入力を送れば共有できるか、全く共有できないかという二択で考えた。実際には固定文脈が共有され、新規から新規と、継続後の新規とで観測が異なった。したがって「新規セッションにはキャッシュ利得がない」という一般的な説明は採らない。比較実験ではその条件依存性を踏まえ、`nonce` と実行順の記録を用いた。

7.2 履歴の増大と書き直しの関係

M6 では、継承履歴が増えても後続ターンの `cache write` が新規追加分で一定の系列を得た。履歴が長いことと、毎回その全量を書き直すことは同じではない。M7 は追加プロンプト量だけで初回書き直しを説明する見立てでも支持しなかった。一方、履歴を `cache read` で繰り返し処理する費用は残る。機構の一部を観測できたことから、仕事全体の正味利益までを推定しない。

7.3 予算付きレーンを成果としない

履歴が予算を超えたら新規に切り替える C' も検討したが、パイロットでは継続の費用増加が観測され、独立した本実験アームとして採用しなかった。少数の観測から厳密な損益分岐 H^* を同定した、あるいは C' が一般に B と同じ動作になると証明した、とは言わない。適応スケジューラ D 、助言注入、他方式の再帰的協調も測定していない。「改善候補を考えた」ことを「改善を実証した」ことへ置き換えない。

8 妥当性と適用範囲の限界

8.1 対象の狭さと仕様への依存

単一言語・単一リポジトリ・単一モデル識別子・単一 CLI 版であり、仕様の作成・選定には著者の判断がある。7 仕様の反復は、7 つの独立したシステムへの再現実験ではない。特に CT 仕様は 1 件なので、同じグラフ形状の他の課題に同じ費用差が出るとは言えない。実運用における CT の出現頻度も測っていない。

8.2 複合介入と観測できない状態

セッション継続と実行順制約を同時に変えているため、観測した費用差を原因別に分解できない。初期 `cache read` との対応は、内部キャッシュ機構の因果的な同定ではない。`nonce` で全共有を遮断した保証はなく、固定文脈やプロバイダ状態は残る。プローブと本実験の CLI 版も違うため、小規模プローブの結果を本実験の全境界へそのまま移さない。

8.3 順序・運用変更と統計上の仮定

49 対ずつの先行方式の均衡は確認したが、アーム順は反復に依存し、時間変動との独立性を保証しない。A-5/A-6 などの変更、再実行、利用上限への対応がある。感度分析が集約結論を変えなくても、あらゆる交絡を排除したとは言えない。符号反転検定とブートストラップは、対の独立性や交換可能性の仮定に依存する。近接した反復のサービス状態に相関があれば、不確実性が過小評価されうる。

8.4 モデル近似と較正の量

較正は各仕様 1 反復なので、探索量や履歴量の分散を十分に推定できない。最初の編集以前を探索とする定義では、編集後の再探索を作業側に分類する。 $\hat{S}$ は最小観測文脈であり、固定システム文脈そのものの測定値ではない。継続後の出力量、ツール行動、履歴の累積を十分に予測していない。探索／作業の境界をずらす感度分析は実施しておらず、固定予測を今回の結果に合わせて更新していない。

8.5 品質の範囲と訂正の位置づけ

受入テストは機械的だが、全ての品質属性を表さない。10 ポイントの許容差は運用判断で、品質が同じであることを示す基準ではない。二値の不一致が少ないため、補正後の BCa でも小標本・離散分布の限界がある。正確二項上限を用いる補助解析も、独立で共通の不利益確率を持つという仮定の下での確認である。

改稿時に H1 の尺度と H3 の統計量の実装不一致が判明したことは、本研究の再現性の限界でもある。合成テストが存在していたことだけでは、呼び出し側が正しい統計量を渡すことまで保証できなかった。訂正後は、通過率差、対ごとの通貨尺度の変更、等値予測の扱いを検証する合成ケースを追加した。訂正後の値を最初から得ていたかのように記載しない。

8.6 装置を再配布しないことによる再現の非対称

本研究の再現可能性は分析側と実験側で非対称である。196 ランの計測データは同梱するため、集計・判定・図表の再実行は本パッケージだけで完結する。一方、実験を回した実行フレームワークと、実験された対象リポジトリはいずれもライセンスが設定されておらず、本研究では再配布しないと決めた。公開物に含まれるのは実験手順と仕様の記述であって、装置そのものではない。

したがって第三者は、本稿の数値を独立に再取得して検証することができない。検証できるのは、同梱データから同じ結論が導かれるかまでである。判定規則を機械適用した点と事前指定文書の保存は、分析者の自由度を封じるものであり、データ生成そのものの独立検証の代わりにはならない。これは変更不能な商用 API を私有リポジトリ上で測る設計に共通する制約だが、共通であることは制約を消さないため、主張の範囲をここに留める。

8.7 非盲検と公開前の証拠

研究の設計者はパイロットの方向を見ており、C' の着想も観測後である。B 側だけで較正したことは C/B 比への直接の適合を避けるが、完全な盲検ではない。またローカル Git の日時

は改変可能であり、それだけで独立した事前登録時刻を証明しない。本稿は保存された事前指定文書と変更履歴を開示するが、外部の登録・公開を完了したとは述べてない。

9 結論

商用コーディングエージェント CLI について、短いプローブでキャッシュ挙動を確認し、費用モデルと特定のセッション継続方式を評価した。履歴のキャッシュ読みは観測できたが、それだけで作業全体の費用削減を保証するものではなかった。

1 リポジトリ・7 仕様・98 対の比較では、C の相対費用差中央値は +5.51% で、事前指定した 20% 削減の達成条件を満たさなかった。宣言済みの相対尺度に実装を訂正した 95% 区間は [-3.18, +10.12]% である。時間と受入全通過率は、明記した許容差についての非劣性基準を満たした。モデル予測は大きさの許容誤差だけでは方向の判断を保証せず、全仕様での方向一致を満たさなかった。

CT 仕様で大きな費用増加が観測され、それを除くと集約中央値の符号も変わった。これは課題構成への依存を示すが、直列化単独の因果効果や、CT 一般の費用増加を確立しない。今後は、スケジュールを揃えた継続有無の比較、複数リポジトリ・CLI 版での再現、事後観測を用いた切り替えなどを独立した研究で検証する必要がある。現時点では、会話継続全般を推奨・否定する一般則や、未評価の適応方式の優位性を主張しない。

A 分析訂正の履歴

項目	保存した旧結果・旧説明	本稿の扱い
H1 の尺度	USD 差の $p = 0.0047$ と区間	分析前文書の相対差へ訂正。 $p = 0.0013$ 、区間 [-3.18, +10.12]%。 旧値は表4に併記
H3 の統計量	中央値の下限 0.00 ポイント	通過率差の平均へ訂正。下限-5.10 ポイント。基準通過は維持
H4b の方向	等値を含む二値判定を「2 つの値下がり予測ミス」と説明	旧コードの不一致集合と 3 値記述を分ける。いずれも全一致せず
予測の偏り	7/7 で過大予測	表と計算に一致する 6/7 へ訂正
TTL の記述	約 10.2 分まで warm と断定	612.1 秒・916.9 秒の保存分類は ambiguous。寿命の確定値を撤回
失敗時の継続追加の区間	必ずセッション破棄 未掲載	実装にその一律保証がないため撤回 仕様内復元抽出の percentile 区間、受入の保守的二項下限を事後分析として掲載

表 7: 2026 年 9 月 4 日の統合時の訂正。原データと判定閾値は変更していない。

旧版は paper/revision/original/ に保存し、ファイルごとの SHA-256 を manifest.json に記録した。凍結結果は引き続き analysis/e2_results_main.json、訂正結果は paper/revision/results.json である。主要結論が変わらないことと、計算・説明に誤りがなかったことは同じではない。本稿は誤りを訂正し、旧結果の再現可能性を保つ。

B 機構測定の記録と未確定事項

probes/resume_probe_results.jsonl には初期試行と有効な反復をともに保存している。M1 の一意接頭辞付き 3 反復は、2026 年 7 月 31 日 02:09 台の init/resume 対である。その前の共有接頭辞による試行と混ぜない。M4 の 3 対では cache write/read 内訳が一致する一方、出力を含む総費用比には 1.082 の例がある。したがって「総費用が全て完全一致」ではなく、キャッシュ内訳の一致として報告する。

長間隔プローブは 612.1 秒と 916.9 秒の 2 件で、保存判定はどちらも ambiguous である。612.1 秒の cache write/read は 29,572/15,912、916.9 秒では 29,417/15,912 だった。固定文脈の一部が残る状態と履歴全量の再利用を区別でき、厳密な保持時間の推定値としては扱わない。初期の約 6.4 分後の読み取り観測もあるが、同一セッションの継続測定で寿命更新の影響を含む。warm の機構チェックを満たすことから、H4a の長間隔・切替の全条件まで合格したとはしない。

m6_results.jsonl と m7_results.jsonl は追加量と初期 cache read の所見を支える。M7 の 12 測定と、M6 を含めた初回状態の例を混在した表を同じ標本数として扱わない。提供側のブレイクポイント配置などの内部原因は、このログだけから確定できない。

C 追加分析の定義

固定 7 仕様の層別ブートストラップでは、各仕様から 14 対を復元抽出し、合計 98 対の相対差中央値を計算する。これを 10,000 回繰り返す、線形補間した 2.5・97.5 百分位を用いた。仕様そのものを無作為に採取した設計ではないので、仕様数を増やす母集団推論とは区別する。

受入について、 $q = P(B\text{成功}, C\text{失敗})$ 、 $g = P(B\text{失敗}, C\text{成功})$ なら、通過率差は $g - q \geq -q$ である。観測した不利益不一致は 1/98 なので、二項分布の片側 90% 正確上限 U_q [1] を用いると、 $-U_q$ が保守的な通過率差下限となる。 $\sum_{j=0}^1 \binom{98}{j} U_q^j (1 - U_q)^{98-j} = 0.10$ を解き、下限-3.91 ポイントを得た。各対に共通の不利益確率と独立性を仮定する補助確認であり、仕様別の受入品質を保証しない。

D 再現方法と成果物

統合本文の正本は paper/main.tex である。数値・表・図は訂正結果 JSON と同じデータセットから生成する。

生成スクリプトは paper/render_assets.py である。原稿と図表を作り直す手順は次のとおり。

```
python3 analysis/revision_audit.py
.venv-figs/bin/python paper/render_assets.py
bash paper/build.sh
```

分析訂正は標準 Python ライブラリのみ、作図は NumPy/Matplotlib、PDF 生成は XeLaTeX 互換の Tectonic と日本語 Noto CJK フォントを用いる。ビルドスクリプトは取得済み Tectonic または PATH 上の実行ファイルを使う。未取得の TeX 資材には初回のネットワークアクセスが必要となる。paper/BUILD.md に環境とファイルの役割を記す。

分析は保存データだけで再実行できるが、本実験の再実行には商用サービスへのアクセス、実行フレームワーク、対象リポジトリの固定版が必要である。本原稿のソース一式だけで 196 ランを再取得できるとは主張しない。著者表示・所属・連絡先、公開リポジトリ URL、DOI、投稿先識別子は未確定のため作成していない。ソースと PDF をローカルに作成したことを、外部公開の完了と同一視しない。

参考文献

- [1] C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934. <https://doi.org/10.1093/biomet/26.4.404>.
- [2] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993. <https://doi.org/10.1007/978-1-4899-4541-9>.
- [3] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. Parrot: Efficient serving of LLM-based applications with semantic variable. arXiv:2405.19888, version 1, 2024. <https://arxiv.org/abs/2405.19888v1>.
- [4] Elias Lumer, Faheem Nizar, Akshaya Jangiti, Kevin Frank, Anmol Gulati, Mandar Phadate, and Vamse Kumar Subbiah. Don’t break the cache: An evaluation of prompt caching for long-horizon agentic tasks. arXiv:2601.06007, version 2, 2026. <https://arxiv.org/abs/2601.06007v2>.
- [5] Belinda Phipson and Gordon K. Smyth. Permutation p -values should never be zero: Calculating exact p -values when permutations are randomly drawn. *Statistical Applications in Genetics and Molecular Biology*, 9(1):Article 39, 2010. <https://doi.org/10.2202/1544-6115.1585>.
- [6] Cosmo Santoni. Contextual memory virtualisation: DAG-based state management and structurally lossless trimming for LLM agents. arXiv:2602.22402, version 1, 2026. <https://arxiv.org/abs/2602.22402v1>.
- [7] Michelle Vaccaro. Preregistration for experiments with AI agents. arXiv:2606.11217, version 1, 2026. <https://arxiv.org/abs/2606.11217v1>.
- [8] Xu Yang, Lunyu Nie, Ethan Chandra, Stanislav Gannutin, Fangru Lin, and Swarat Chaudhuri. When parallelism pays off: Cohesion-aware task partitioning for multi-agent coding. arXiv:2606.00953, version 1, 2026. <https://arxiv.org/abs/2606.00953v1>.
- [9] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. arXiv:2312.07104, version 2, 2024. <https://arxiv.org/abs/2312.07104v2>.